Exception Handling

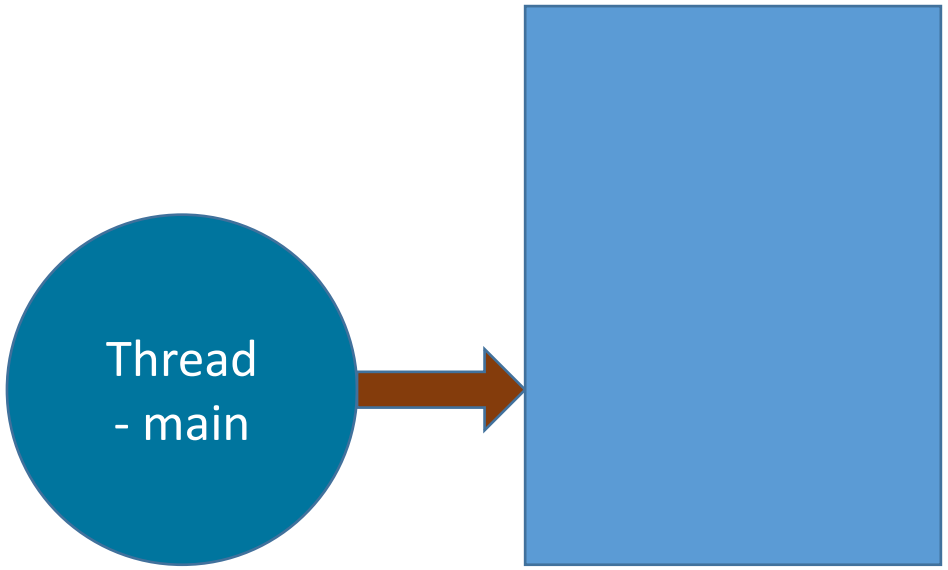
The process of responding to unwanted or unexpected events when a computer program runs

1. Introduction
2. Runtime stack mechanism
3. Default exception handling in java
4. Exception hierarchy
5. Customized exception handling by try catch
6. Control flow in try catch
7. Methods to print exception information
8. Try with multiple catch blocks
9. Finally
10. Difference between final, finally, finalize
11. Control flow in try catch finally
12. Control flow in nested try catch finally
13. Various possible combinations of try catch finally

14. throw keyword
15. throws keyword
16. Exception handling keywords summary
17. Various possible compile time errors in exception handling
18. Customized exceptions
19. Top-10 exceptions
20. 1.7 Version Enhancements
21. try with resources
22. multi catch block
23. Exception Propagation
24. Rethrowing an Exception

```
class Exc{
 public static void main(String args[]){
  int d=0;
  int a=42/d;
 }
}
```

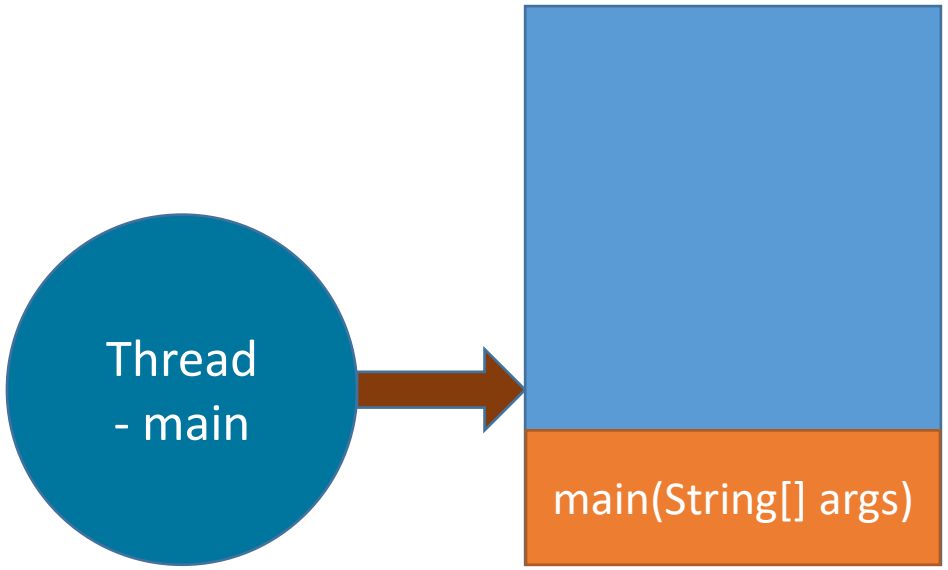
```
class A{
 public static void main(String args[]){
  System.out.println("main method");
 }
}
```



Runtime stack

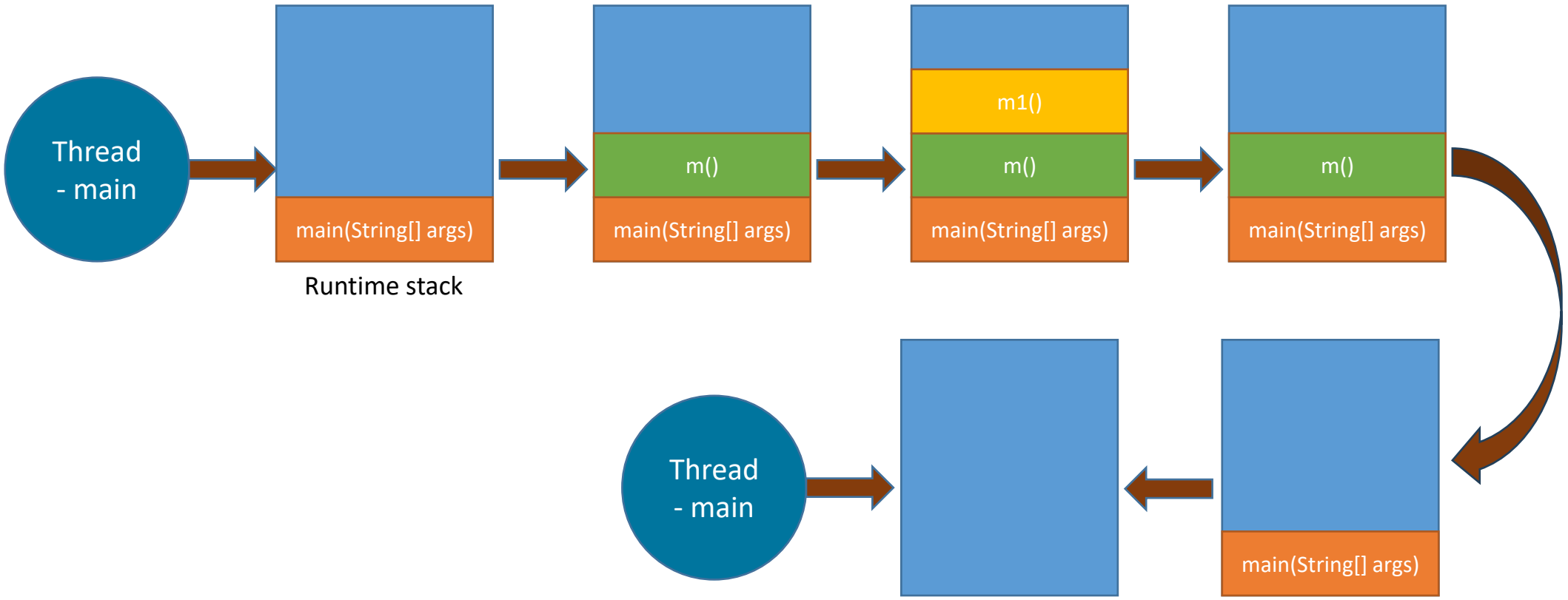
main(String[] args)

Stack frame

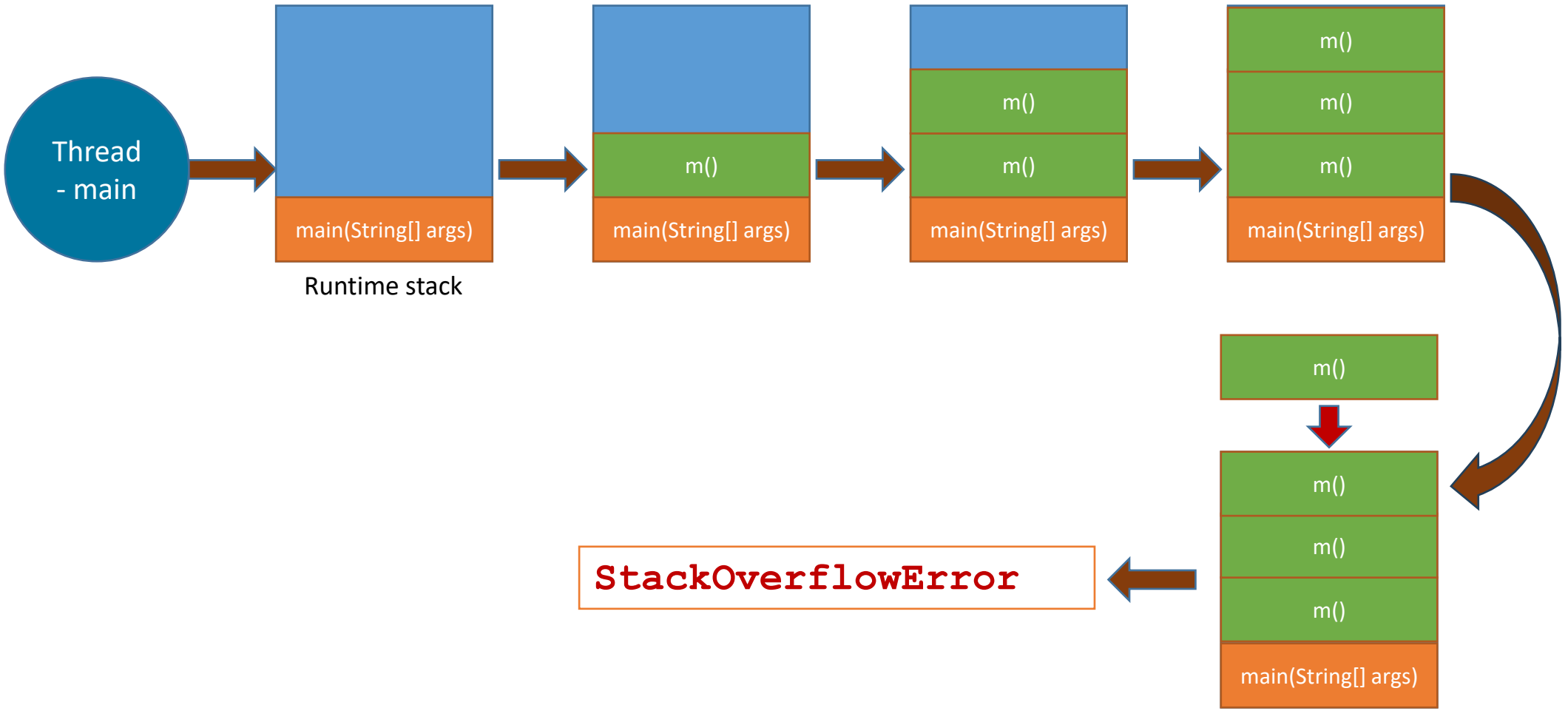


Runtime stack

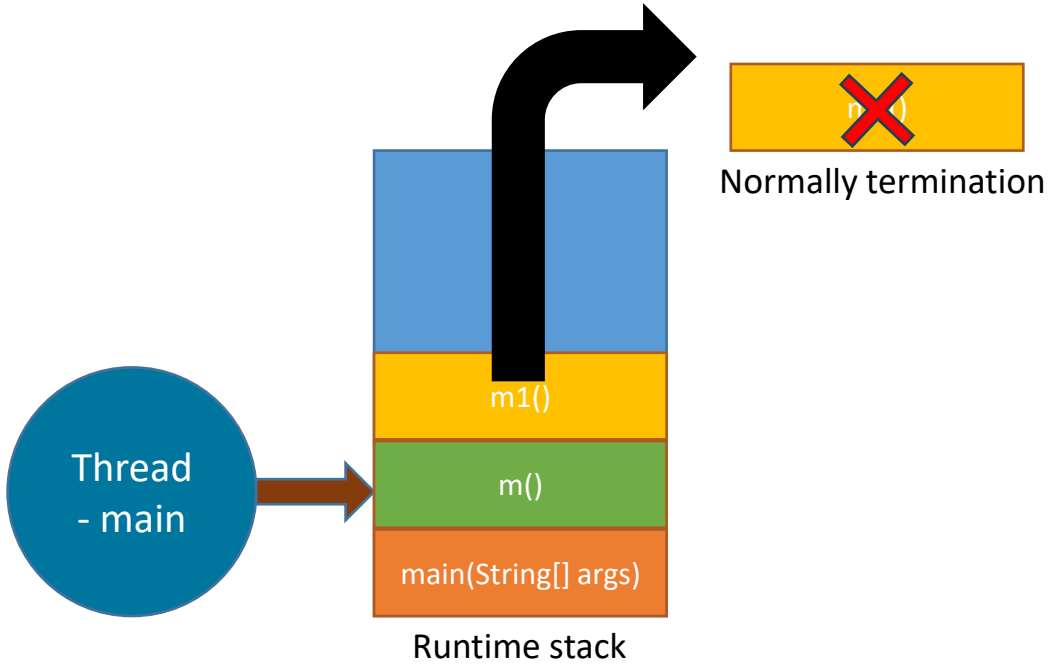
```
class A{
    static void m(){
        System.out.println("m");
        m1();
    }

    static void m1(){
        System.out.println("m1");
    }

    public static void main(String[] args) {
        m();
    }
}
```



```
class A{
    static void m(){
        System.out.println("m");
        m();
    }

    public static void main(String[] args) {
        m();
    }
}
```

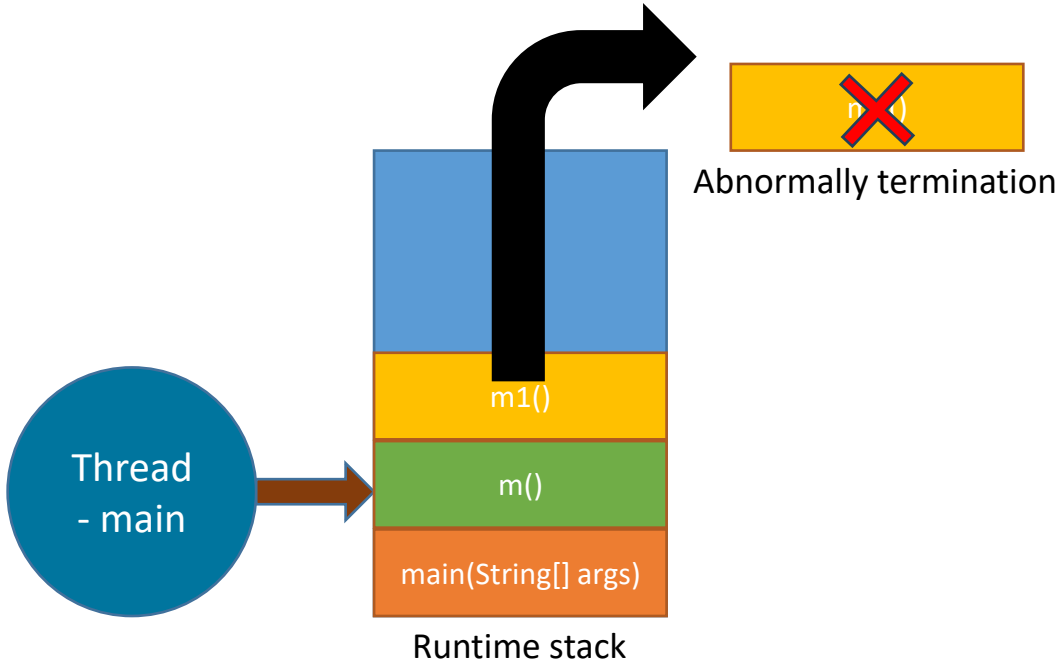


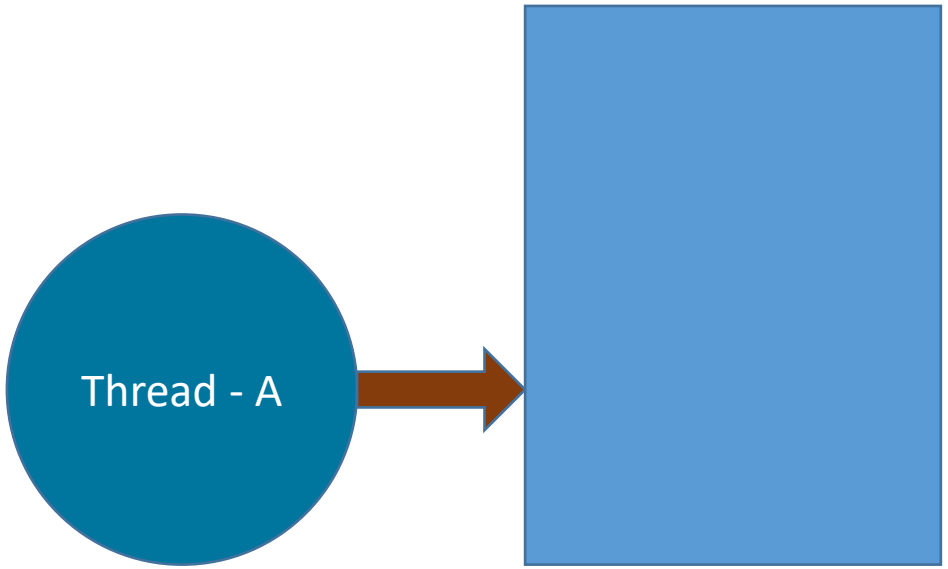
```
class A{
    static void m(){
        System.out.println("m");
        m1();
    }

    static void m1(){
        System.out.println("m1");
    }

    public static void main(String[] args) {
        m();
    }
}
```



```
class A{
    static void m(){
        System.out.println("m");
        m1();
    }

    static void m1(){
        System.out.println("m1");

        int i = 10/0;
        System.out.println("m1");

        System.out.println("m1");

    }

    public static void main(String[] args) {
        m();
    }
}
```

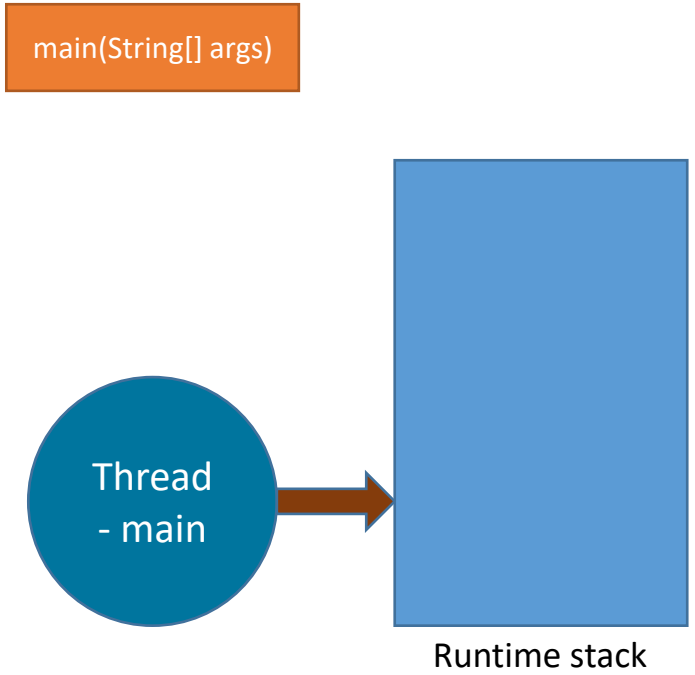
Unwanted unexpected event





Runtime stack

```
class A{
 public static void main(String args[]){
  int d=0;
  int a=10/d;

   System.out.println(a);
 }
}
```



```
class Exc1 {
 public static void main(String args[]){
  int d=0;
  System.out.println("before");
  int a=42/d;
  System.out.println("after");
 }
}
```

```
class Exc2 {
 public static void main(String args[]){
  String months[]=new String[12];
  System.out.println("before");
  months[12]="December";
  System.out.println("after");
 }
}
```

```
class Exc3 {
 static void m(){
  int x=0;
  int d=42/x;
  System.out.println("d :"+d);
 }
 public static void main(String args[]){
  System.out.println("before");
  m();
  System.out.println("after");
 }
}
```

```
class TryCatch {
 public static void main(String args[]){
  int d=0,x=0;
  System.out.println("before");
  try{
   x=42/d;
  }catch(ArithmeticException e){
   System.out.println("Divide by Zero");
   x=0;
  }
  System.out.println("after");
 }
}
```

```
class TryCatch {
 public static void main(String args[]){
  int d=0,x=0;
  System.out.println("before");
  try{
   x=42/d;
  }catch(ArithmeticException e){
   System.out.println(e);
   x=0;
  }
  System.out.println("after");
 }
}
```

```
class Exc4 {
 public static void main(String args[]){
  String months[]=new String[12];
  System.out.println("before");
  try{
   months[12]="December";
  }catch(ArrayIndexOutOfBoundsException e){
   System.out.println(e);
  }
  System.out.println("after");
 }
}
```

```
class Exc5 {
 public static void main(String args[]){
  String name="123456789V";
  System.out.println("before");
  try{
   char ch=name.charAt(10);
  }catch(StringIndexOutOfBoundsException e){
   System.out.println("Runtime Error :"+e);
  }
  System.out.println("after");
 }
}
```

```
class Exc6 {
 public static void main(String args[]){
  String name="123456789V";
  System.out.println("before");
  try{
   char ch=name.charAt(10);
  }catch(IndexOutOfBoundsException e){
   System.out.println("Runtime Error :"+e);
  }
  System.out.println("after");
 }
}
```

```
class Exc7 {
 public static void main(String args[]){
  String name="123456789V";
  System.out.println("before");
  try{
   char ch=name.charAt(10);
  }catch(RuntimeException e){
   System.out.println("Runtime Error :"+e);
  }
  System.out.println("after");
 }
}
```

```
class Exc8 {
 public static void main(String args[]){
  String name="123456789V";
  System.out.println("before");
  try{
   char ch=name.charAt(10);
  }catch(Exception e){
   System.out.println("Runtime Error :"+e);
  }
  System.out.println("after");
 }
}
```

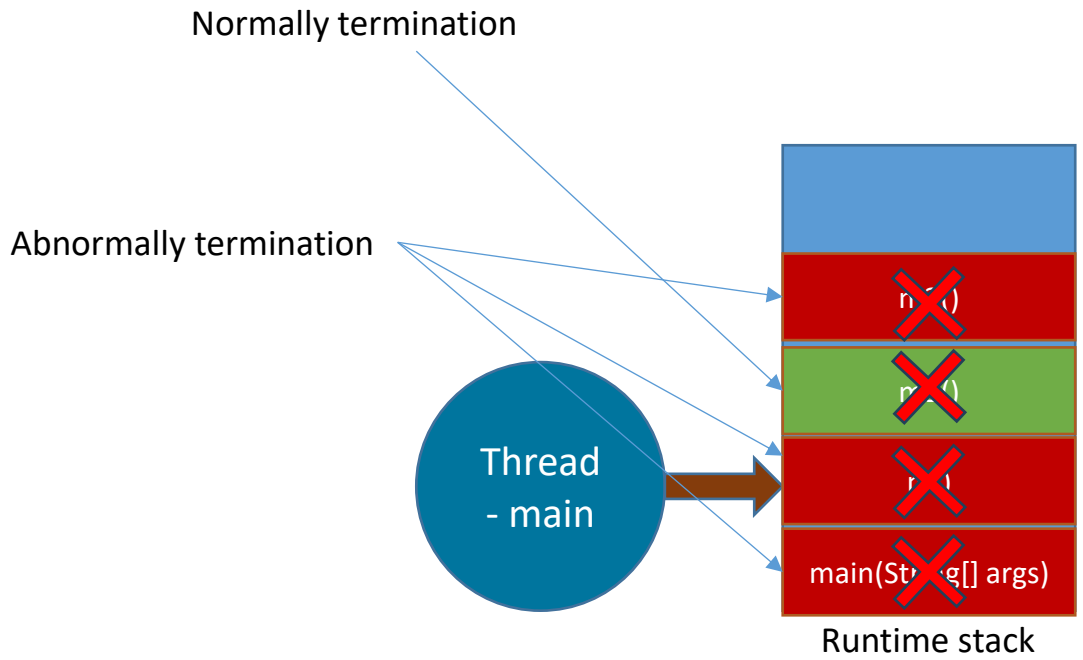
```
class Exc9 {
 public static void main(String args[]){
  String name="123456789V";
  System.out.println("before");
  try{
   char ch=name.charAt(10);
  }catch(Throwable e){
   System.out.println("Runtime Error :"+e);
  }
  System.out.println("after");
 }
}
```

Default Exception Handling

```
class A{
    static void m(){
        System.out.println("m");
        m1();
    }

    static void m1(){
        System.out.println("m1");
    }

    public static void main(String[] args) {
        m();
    }
}
```

```
class A{
    static void m(){
        System.out.println("m");
        m1();
    }

    static void m1(){
        System.out.println(10/0);
    }

    public static void main(String[] args) {
        m();
    }
}
```

```
class A{
    static void m(){
        System.out.println("m");
        m1();
         m2();
    }

    static void m1(){
        System.out.println("m1");
    }

    static void m2(){
        System.out.println(10/0);
    }


    public static void main(String[] args) {
        m();
         System.out.println("main end");
    }
}
```



Output:

m

m1

Exception in thread "main" java.lang.ArithmeticException: / by zero

```
class A {

    static void m() {
        System.out.println("a");
        m1();
        System.out.println("b");
    }

    static void m1() {
        System.out.println(10 / 0);
    }

    public static void main(String[] args) {
        m();
        System.out.println("main");
    }
}
```

1. If an exception is raised inside any method then that method is responsible to create an Exception object with the following information.
   - Name of the exception.
   - Description of the exception.
   - Location of the exception.(StackTrace)

2. After creating the Exception object, the method passes it to the JVM.

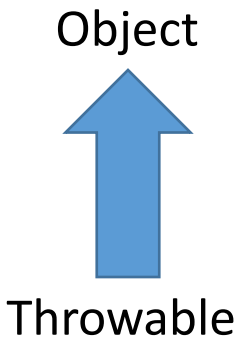
3. JVM checks whether the method contains any exception handling code or not. If method won't contain any handling code then JVM terminates that method abnormally and removes corresponding entry form the stack.

4. JVM identifies the caller method and checks whether the caller method contain any handling code or not. If the caller method also does not contain handling code then JVM terminates that caller method also abnormally and removes corresponding entry from the stack.

5. This process will be continued until main() method and if the main() method also doesn't contain any exception handling code then JVM terminates main() method also and removes corresponding entry from the stack.

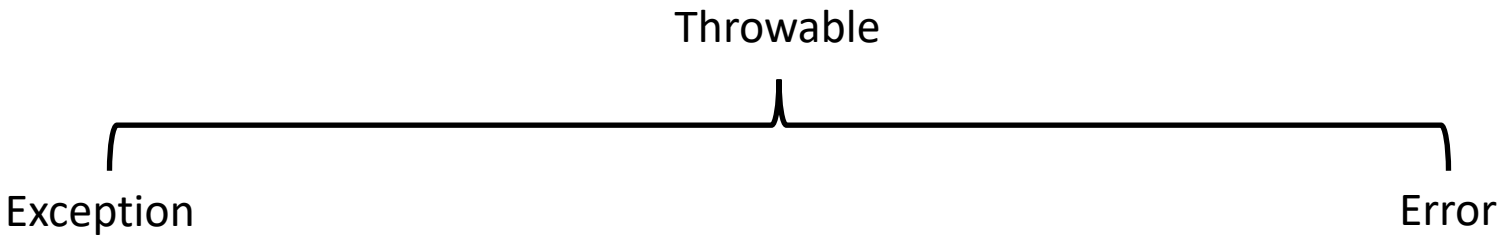
6. Then JVM handovers the responsibility of exception handling to the default exception handler.

Exception Hierarchy



- Throwable acts as a root for exception hierarchy.

- Throwable class contains the following two child classes.
  - Exception
  - Error



Exception:
Most of the cases exceptions are caused by our program and these are recoverable.
Ex : If FileNotFoundException occurs then we can use local file and we can continue rest of the program execution normally.

Error:
Most of the cases errors are not caused by our program these are due to lack of system resources and these are non-recoverable.
Ex :If OutOfMemoryError occurs being a programmer we can't do anything the program will be terminated abnormally.
System Admin or Server Admin is responsible to raise/increase heap memory.

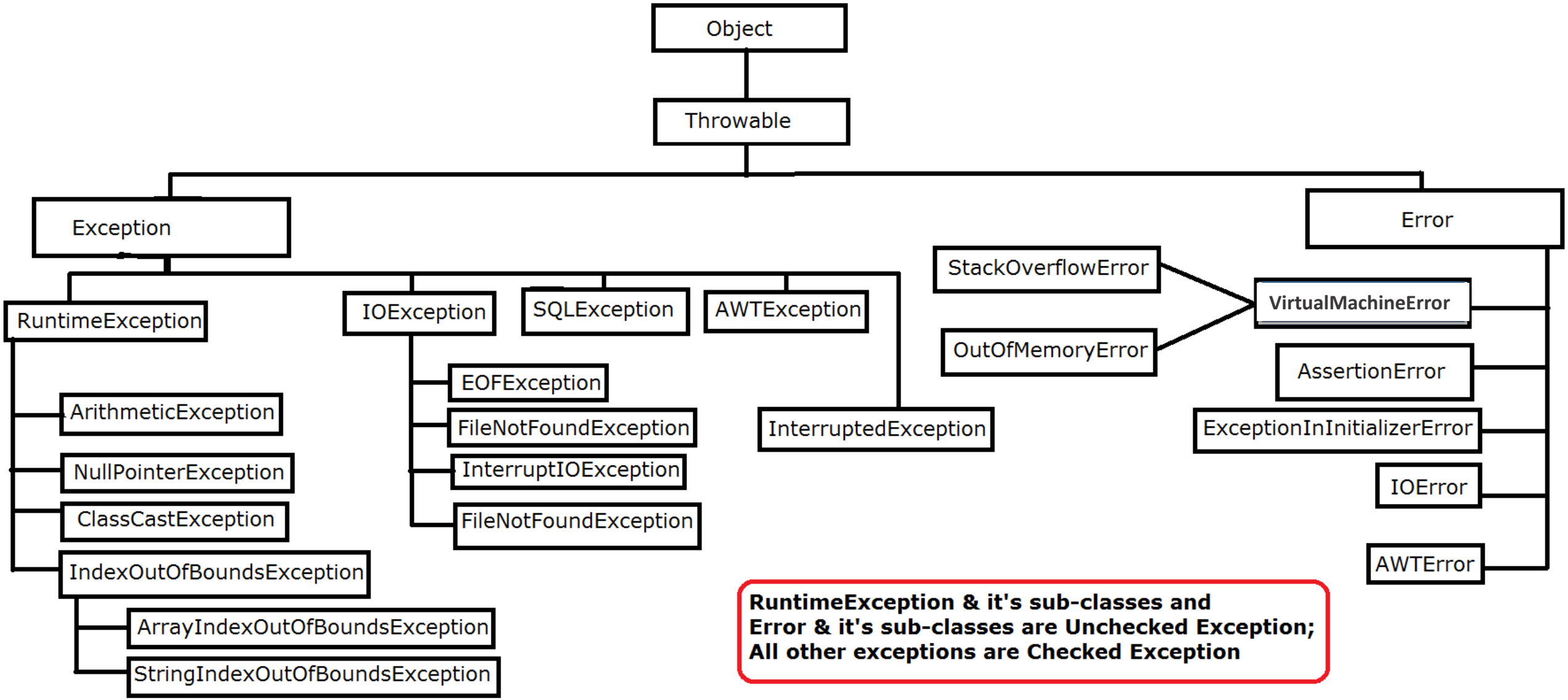
Checked Vs Unchecked Exceptions

- The exceptions which are checked by the compiler whether programmer handling or not, for smooth execution of the program at runtime, are called checked exceptions

- The exceptions which are not checked by the compiler whether programmer handing or not ,are called unchecked exceptions.

Note:
RuntimeException and its child classes, Error and its child classes are unchecked and all the remaining are considered as checked exceptions.

Note:
Whether exception is checked or unchecked compulsory it should occurs at runtime only and there is no chance of occurring any exception at compile time.



Customized Exception Handling by using try-catch

- It is highly recommended to handle exceptions.
- In our program the code which may raise exception is called **risky code**, we must place **risky code** inside try block and the corresponding handling code inside catch block.

```
try{

 Risky code

}catch(Exception e){

 Handling code

}
```

Control flow in try catch

```
try{

    statement1;
    statement2;
    statement3;

}catch(X e) {

    statement4;

}

statement5;
```

Case Study

Case 1:
If there is no exception.
1, 2, 3, 5 normal termination.

Case 2:
if an exception raised at statement 2 and corresponding catch block matched
1, 4, 5 normal termination.

Case 3:
if an exception raised at statement 2 but the corresponding catch block not matched
1 followed by abnormal termination.

Case 4:
if an exception raised at statement 4 or statement 5 then it's always
abnormal termination of the program.

Note:

- Within the try block if anywhere an exception raised then rest of the try block won't be executed even though we handled that exception. Hence, we must place/take only risk code inside try block and length of the try block should be as less as possible.

- If any statement which raises an exception, and it is not part of any try block then it is always abnormal termination of the program.

- There may be a chance of raising an exception inside catch and finally blocks also in addition to try block.

```
class A {

    static void m() {
        System.out.println("a");
        m1();
        System.out.println("b");
    }

    static void m1() {
        System.out.println(10/0);
    }

    public static void main(String[] args) {
        m();
        System.out.println("main");
    }
}
```

```
class A {

    static void m() {
        System.out.println("a");
        m1();
        System.out.println("b");
    }

    static void m1() {
        try{
            System.out.println(10/0);
        }catch(Exception e){
            System.out.println("I got the error");
        }
    }

    public static void main(String[] args) {
        m();
        System.out.println("main");
    }
}
```

```
class A {

    static void m() {
        System.out.println("a");
        try {
            m1();
        } catch (Exception e) {
            System.out.println("I got the error");
        }
        System.out.println("b");
    }

    static void m1() {
        System.out.println(10/0);
    }

    public static void main(String[] args) {
        m();
        System.out.println("main");
    }
}
```

```
class A {

    static void m() {
        System.out.println("a");
        m1();
        System.out.println("b");
    }

    static void m1() {
        System.out.println(10 / 0);

    }

    public static void main(String[] args) {
        try {
            m();
        } catch (Exception e) {
        }
        System.out.println("main");
    }
}
```

Customized Exception Handling by using try-catch

- It is highly recommended to handle exceptions.
- In our program the code which may raise exception is called **risky code**, we must place **risky code** inside try block and the corresponding handling code inside catch block.

```
try{

 Risky code

}catch(Exception e){

 Handling code

}
```

Various methods to print exception information

Throwable class defines the following methods to print exception information to the console.

printStackTrace():
This method prints exception information in the following format.
- Name of the exception: description of exception
- Stack trace

toString():
This method prints exception information in the following format.
- Name of the exception: description of exception

getMessage():
This method returns only description of the exception.
- Description

Note: Default exception handler internally uses **printStackTrace()** method to print exception information to the console.

Try with multiple catch blocks

The way of handling an exception is varied from exception to exception. Hence for every exception type it is recommended to take a separate catch block. That is try with multiple catch blocks is possible and recommended to use.

```
try{

        statement;

}catch(Exception e){

        default handler

}
```

This approach is not recommended because for any type of Exception we are using the same catch block.

Try with multiple catch blocks

```
try{
        statement;
}catch(FileNotFoundException e){
        use local file
}catch(ArithmeticException e){
        perform these Arithmetic operations
}catch(SQLException e){
        don't use oracle db, use mysqldb
}catch(Exception e){
        default handler
}
```

This approach is highly recommended because for any exception raise we are defining a separate catch block.

Note:

If try with multiple catch blocks present, then order of catch blocks is very important. It should be from child to parent by mistake if we are taking from parent to child then we will get Compile time error.

"exception xxx has already been caught"

```
class Test {

    public static void main(String[] args) {
        try {
            System.out.println(10 / 0);
        } catch (Exception e) {
            e.printStackTrace();
        } catch (ArithmeticException e) {
            e.printStackTrace();
        }
    }
}
```

CE:exception
java.lang.ArithmeticException has already been caught

```
class Test {

    public static void
            main(String[] args) {
        try {
            System.out.println(10 / 0);
        } catch (ArithmeticException e) {
            e.printStackTrace();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Compile successfully.

Finally block

- It is not recommended to take clean up code inside try block because there is no guarantee for the execution of every statement inside a try.
- It is not recommended to place clean up code inside catch block because if there is no exception then catch block won't be executed.
- We require some place to maintain clean up code which should be executed always irrespective of whether exception raised or not raised and whether handled or not handled. Such type of best place is nothing but finally block.
- Hence the main objective of finally block is to maintain cleanup code.

Finally block

- It is not recommended to take clean up code inside try block because there is no guarantee for the execution of every statement inside a try.
- It is not recommended to place clean up code inside catch block because if there is no exception then catch block won't be executed.
- We require some place to maintain clean up code which should be executed always irrespective of whether exception raised or not raised and whether handled or not handled. Such type of best place is nothing but finally block.
- Hence the main objective of finally block is to maintain cleanup code.

```
try{
        risky code
}catch(x e){
        handling code
}finally{
        cleanup code
}
```

The specialty of finally block is it will be executed always irrespective of whether the exception raised or not raised and whether handled or not handled.

Case-1: If there is no Exception

```
class Test {

    public static void main(String[] args) {
        try {
            System.out.println("try block executed");
        } catch (ArithmeticException e) {
            System.out.println("catch block executed");
        } finally {
            System.out.println("finally block executed");
        }
    }
}
```

Output:
try block executed
Finally block executed

Case-2: If an exception raised and the corresponding catch block matched

```
class Test {

    public static void main(String[] args) {
        try {
            System.out.println("try block executed");
            System.out.println(10 / 0);
        } catch (ArithmeticException e) {
            System.out.println("catch block executed");
        } finally {
            System.out.println("finally block executed");
        }
    }
}
```

Output:

Try block executed
Catch block executed
Finally block executed

Case-3: If an exception raised but the corresponding catch block not matched

```
class Test {

    public static void main(String[] args) {
        try {
            System.out.println("try block executed");
            System.out.println(10 / 0);
        } catch (NullPointerException e) {
            System.out.println("catch block executed");
        } finally {
            System.out.println("finally block executed");
        }
    }
}
```

Output:

Try block executed
Finally block executed
Exception in thread "main" java.lang.ArithmeticException: / by zero
atTest.main(Test.java:8)

return Vs finally

Even though return statement present in try or catch blocks first finally will be executed and after that only return statement will be considered.

i.e. finally block dominates return statement.

```
class Test {

    public static void main(String[] args) {
        try {
            System.out.println("try block executed");
            return;
        } catch (ArithmeticException e) {
            System.out.println("catch block executed");
        } finally {
            System.out.println("finally block executed");
        }
    }
}
```

Output:

try block executed
Finally block executed

If return statement present try, catch and finally blocks then finally block return statement will be considered.

```
class Test {

    public static int m1() {
        try {
            System.out.println(10 / 0);
            return 777;
        } catch (ArithmeticException e) {
            return 888;
        } finally {
            return 999;
        }
    }

    public static void main(String[] args) {
        System.out.println(m1());
    }

}
```

finally vs System.exit(0)

There is only one situation where the finally block won't be executed is whenever we are using System.exit(0) method.

When ever we are using System.exit(0) then JVM itself will be shutdown , in this case finally block won't be executed.

```
class Test {

    public static void main(String[] args) {
        try {
            System.out.println("try");
            System.exit(0);
        } catch (ArithmeticException e) {
            System.out.println("catch block executed");
        } finally {
            System.out.println("finally block executed");
        }
    }
}
```

Note :

```
System.exit(0);
```

1. This argument acts as status code. Instead of zero, we can take any integer value.
2. zero means normal termination , non-zero means abnormal termination.
3. This status code is internally used by JVM, whether it is zero or non-zero there is no change in the result.

Difference between final, finally, and finalize

final:

- final is the modifier applicable for classes, methods and variables.
- If a class declared as the final then child class creation is not possible.
- If a method declared as the final then overriding of that method is not possible.
- If a variable declared as the final then reassignment is not possible.

finally:

- finally is the block always associated with try-catch to maintain clean up code which should be executed always irrespective of whether exception raised or not raised and whether handled or not handled.

finalize:

- finalize is a method, always invoked by Garbage Collector just before destroying an object to perform cleanup activities.

Note:

finally block meant for cleanup activities related to try block where as **finalize()** method meant for cleanup activities related to object.

To maintain clean up code **finally** block is recommended over **finalize()** method because we can't expect exact behavior of **GC**.

Control flow in try catch finally

```
try{

        statement1;
        statement2;
        statement3;

}catch(Exception e) {

        statement4;

}finally{

        statement5;
}

statement6;
```

Case Study

Case 1:
If there is no exception. 1, 2, 3, 5, 6 normal termination.

Case 2:
if an exception raised at statement 2 and corresponding catch block matched. 1,4,5,6 normal terminations.

Case 3:
if an exception raised at statement 2 and corresponding catch block is not matched. 1,5 abnormal termination.

Case 4:
if an exception raised at statement 4 then it's always abnormal termination but before the finally block will be executed.

Case 5:
if an exception raised at statement 5 or statement 6 its always abnormal termination.

Nested try catch

```
try{
        statement1;

        try{
                statement2;
        }catch(X e) {
                statement3;
         }

         Statement4;

}catch(Y e) {
        statement5;
}finally{
        statement6;
}
```

Nested try-catch is a way to handle errors in code by placing one try-catch block inside another.

This allows you to manage different types of errors at different levels.

- If an error happens in the inner try block, it can be caught and handled in the inner catch block.
- If that catch block rethrows the error, it can be caught in the outer catch block.

Both inner and outer try-catch blocks can also have finally blocks, which run cleanup code no matter what happens.

While nested try-catch blocks can make error handling better, using too many can make the code harder to read, so it's important to use them wisely. They are helpful when different parts of the code might cause different errors that need specific handling.

```
public class Test {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block");
            try {
                System.out.println("Inner try block");
                int result = 10 / 0;
            } catch (ArithmeticException e) {
                System.out.println("Caught ArithmeticException in inner catch: " + e.getMessage());
            }
        } catch (Exception e) {
            System.out.println("Caught Exception in outer catch: " + e.getMessage());
        }
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block");
            try {
                String str = null;
                System.out.println(str.length());
            } catch (NullPointerException e) {
                System.out.println("Caught NullPointerException in inner catch: " + e.getMessage());
            }
        } catch (Exception e) {
            System.out.println("Caught Exception in outer catch: " + e.getMessage());
        }
    }
}
```

```
public class Test{
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block");
            try {
                System.out.println("Inner try block");
                int[] arr = new int[5];
                System.out.println(arr[10]);
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Caught ArrayIndexOutOfBoundsException in inner catch: " + e.getMessage());
            } finally {
                System.out.println("Inner finally block");
            }
        } catch (Exception e) {
            System.out.println("Caught Exception in outer catch: " + e.getMessage());
        } finally {
            System.out.println("Outer finally block");
        }
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block");
            try {
                int a = 10;
                int b = 0;
                int result = a / b;
            } catch (ArithmeticException e) {
                System.out.println("Caught ArithmeticException in inner catch: " + e.getMessage());
            }
        } catch (Exception e) {
            System.out.println("Caught Exception in outer catch: " + e.getMessage());
        }
    }
}
```

Control flow in Nested try catch finally

```
try{
        statement1;
        statement2;
        statement3;
        try{
                statement4;
                statement5;
                statement6;
        }catch(X e) {
                statement7;
        }finally{
                statement8;
       }
        Statement9;
}catch(Y e) {
        statement10;
}finally{
       statement11;
}

statement12;
```

Case Study

Case 1:
if there is no exception. 1, 2, 3, 4, 5, 6, 8, 9, 11, 12 normal termination.

Case 2:
if an exception raised at statement 2 and corresponding catch block matched 1,10,11,12 normal terminations.

Case 3:
if an exception raised at statement 2 and corresponding catch block is not matched 1, 11 abnormal termination.

Case 4:
if an exception raised at statement 5 and corresponding inner catch has matched 1, 2, 3, 4, 7, 8, 9, 11, 12 normal termination.

Case 5:
if an exception raised at statement 5 and inner catch has not matched but outer catch block has matched. 1, 2, 3, 4, 8, 10, 11, 12 normal termination.

Case 6:
if an exception raised at statement 5 and both inner and outer catch blocks are not matched.
1, 2, 3, 4, 8, 11 abnormal termination.

Case 7:
if an exception raised at statement 7 and the corresponding catch block matched 1, 2, 3, 4, 5, 8, 10, 11, 12 normal termination.

Case 8:
if an exception raised at statement 7 and the corresponding catch block not matched 1, 2, 3, 4, 5, 8, 11 abnormal terminations.

Case 9:
if an exception raised at statement 8 and the corresponding catch block has matched 1, 2, 3, 4, 5, 7, 10, 11,12 normal termination.

Case 10:
if an exception raised at statement 8 and the corresponding catch block not matched 1, 2, 3, 4, 5, 7, 11 abnormal terminations.

Case 11:
if an exception raised at statement 9 and corresponding catch block matched 1, 2, 3, 4, 5, 6, 8,10,11,12 normal termination.

Case 12:
if an exception raised at statement 9 and corresponding catch block not matched 1, 2, 3, 4, 5, 6, 8, 11 abnormal termination.

Case 13:
if an exception raised at statement 10 is always abnormal termination but before that finally block 11 will be executed.

Case 14:
if an exception raised at statement 11 or 12 is always abnormal termination.

Note:

- If we are not entering into the try block then the finally block won't be executed. Once we entered into the try block without executing finally block we can't come out.
- We can take try-catch inside try i.e., nested try-catch is possible
- The most specific exceptions can be handled by using inner try-catch and generalized exceptions can be handle by using outer try-catch.

```
class Test {

    public static void main(String[] args) {
        try {
            System.out.println(10 / 0);
        } catch (ArithmeticException e) {
            System.out.println(10 / 0);
        } finally {
            String s = null;
            System.out.println(s.length());
        }
    }
}
```

Note:
Default exception handler can handle only one exception at a time and that is the most recently raised exception.

Various possible combinations of **try catch finally**

1. Whenever we are writing try block compulsory, we should write either catch or finally.
   i.e. try without catch or finally is invalid.

2. Whenever we are writing catch block compulsory we should write try.
   i.e. catch without try is invalid.

3. Whenever we are writing finally block compulsory we should write try.
   i.e. finally, without try is invalid.

4. In try-catch-finally order is important.

5. With in the try-catch -finally blocks we can take try-catch-finally.
   i.e. nesting of try-catch-finally is possible.

6. For try-catch-finally blocks curly braces are mandatory.

```
try{

}catch(X e){

}
```

```
try{

}catch(X e){

}catch(Y e){

}
```

```
try{

}catch(X e){

}catch(X e){

}
```

CE: exception ArithmeticException has already been caught

```
try{

}catch(X e){

}finally{

}
```

```
try{

}finally{

}
```

```
try{

}
```

CE: 'try' without 'catch', 'finally' or resource declarations

```
catch(X e){

}
```

CE: 'catch' without 'try'

```
finally{

}
```

CE: 'finally' without 'try'

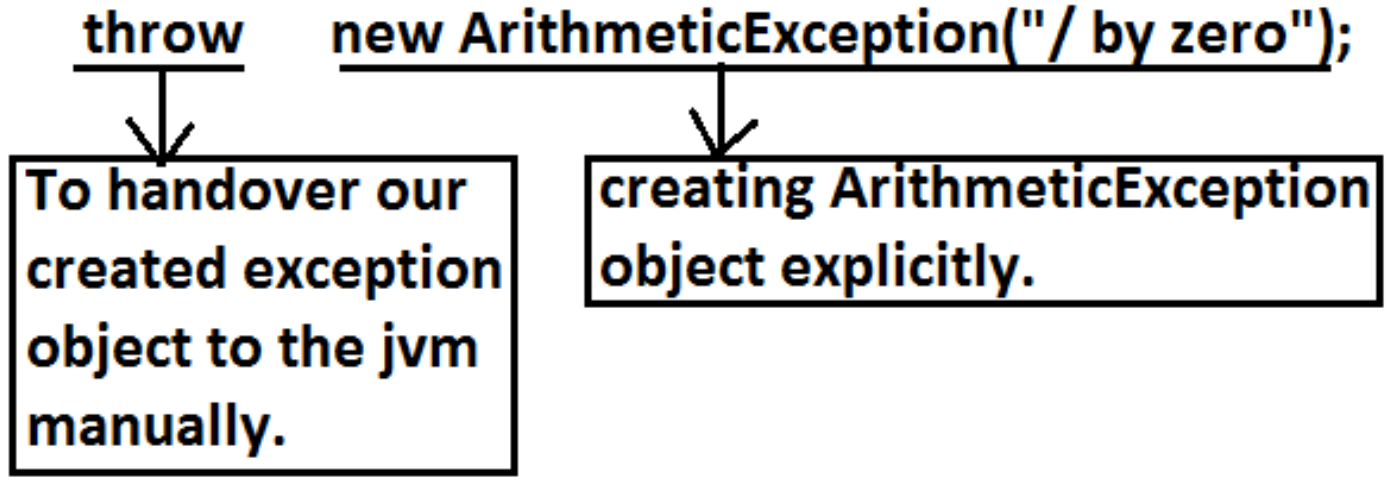
```
try{

}catch(X e){

}finally{

}
```

```
try{

}finally{

}
```

throw statement

Sometimes we can create Exception object explicitly and we can hand over to the JVM manually by using **throw** keyword.

Example:



```
class Test {

   public static void main(String[] args) {
      System.out.println(10 / 0);
   }
}
```

In this case creation of **ArithmeticException** object and handover to the **JVM** will be performed automatically by the **main()** method.

```
class Test {

   public static void main(String[] args) {
      throw new ArithmeticException("/by zero");
   }
}
```

In this case we are creating exception object explicitly and handover to the **JVM** manually.

Note:

In general we can use **throw** keyword for customized exceptions but not for predefined exceptions.

```
class Test3 {

   static ArithmeticException e = new ArithmeticException();

   public static void main(String[] args) {
      throw e;
   }
}
```

```
class Test3 {

   static ArithmeticException e;

   public static void main(String[] args) {
      throw e;
   }
}
```

Case 2:

After throw statement we can't take any statement directly otherwise we will get compile time error saying unreachable statement.

```
class Test3 {

    public static void main(String[] args) {
        System.out.println(10 / 0);
        System.out.println("hello");
    }
}
```

```
class Test3 {

    public static void main(String[] args) {
        throw new ArithmeticException("/ by zero");
        System.out.println("hello");
    }
}
```

unreachable statement

Case 3:

We can use throw keyword only for Throwable types otherwise we will get compile time error saying incompatible types.

```
class Test3 {

    public static void main(String[] args) {
        throw new Test3();
    }
}
```

```
class Test3 extends RuntimeException {

    public static void main(String[] args) {
        throw new Test3();
    }
}
```

throws statement

In our program if there is any chance of raising checked exception then compulsory we should handle either by try catch or by throws keyword otherwise the code won't compile.

```
import java.io.*;

class Test3 {

    public static void main(String[] args) {
        PrintWriter out = new PrintWriter("abc.txt");
        out.println("hello");
    }
}
```

Note :

- Hence the main objective of "throws" keyword is to delegate the responsibility of exception handling to the caller method.

- "throws" keyword required only checked exceptions. Usage of throws for unchecked exception there is no use.

- "throws" keyword required only to convince complier. Usage of throws keyword doesn't prevent abnormal termination of the program.

```
public class Test {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;

        if (b == 0) {
            throw new ArithmeticException("Division by zero is not allowed.");
        } else {
            System.out.println(a / b);
        }
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        String str = null;

        if (str == null) {
            throw new NullPointerException("String cannot be null.");
        } else {
            System.out.println(str.length());
        }
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        int age = -5;

        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative.");
        } else {
            System.out.println("Your age is: " + age);
        }
    }
}
```

```
class Test {

    public static void m1() throws InterruptedException {
        Thread.sleep(5000);
    }

    public static void m() throws InterruptedException {
        m1();
    }

    public static void main(String[] args) throws InterruptedException {
        m();
    }

}
```

1.7 Version Enhancements

try catch

```
try{
        //risky code

}catch(X e){

}
```

try with resources

```
try (     //risky code        ){

}catch(X e){

}
```

```
class Test {

    public static void main(String[] args) throws IOException {
        BufferedReader br = null;
        try {
            br = new BufferedReader(new FileReader("abc.txt"));
            String line = br.readLine();
            while ((line != null) && (!line.isEmpty())) {
                System.out.println(line);
                line = br.readLine();
            }
        } catch (IOException e) {
            e.getStackTrace();
        } finally {
            if (br != null) {
                br.close();
            }
        }
    }
}
```

problems in this approach :

- Compulsory programmer is required to close all opened resources with increases the complexity of the programming

- Compulsory we should write finally block explicitly which increases length of the code and reviews readability.

To overcome these problems Oracle People introduced "**try with resources**" in **1.7** version.

The main advantage of "try with resources" is the resources which are opened as part of try block will be closed automatically.

Once the control reaches end of the try block either normally or abnormally and hence we are not required to close explicitly so that the complexity of programming will be reduced. It is not required to write finally block explicitly and hence length of the code will be reduced and readability will be improved.

Any object that implements **java.lang.AutoCloseable**, which includes all objects which implement **java.io.Closeable**, can be used as a resource.

```
try(BufferedReader br=new BufferedReader(new FileReader("abc.txt"))){
```

*use be based on our requirement, **br** will be closed automatically ,Onec control reaches end of try either normally or abnormally and we are not required to close explicitly*

```
}catch(IOException e) {

// handling code

}
```

1. We can declare any no of resources but all these resources should be seperated with ;(semicolon)

```
try(R1 ; R2 ; R3){
-------------
-------------
}
```

2. All resources should be **AutoCloseable** resources. A resource is said to be auto closable if and only if the corresponding class implements the **java.lang.AutoCloseable** interface either directly or indirectly. All database related, network related and file io related resources already implemented **AutoCloseable** interface. Being a programmer we should aware and we are not required to do anything extra.

3. All resource reference variables are implicitly final and hence we can't perform reassignment with in the try block.

```
try(BufferedReader br=new BufferedReader(new FileReader("abc.txt"))) {
        br=new BufferedReader(new FileReader("abc.txt"));
}
```

output :
CE : Can't reassign a value to final variable br

4. Until 1.6 version try should be followed by either catch or finally but 1.7 version we can take only try with resource without catch or finally.

```
try(R){

}
```

5. The main advantage of "try with resources" is finally block will become dummy because we are not required to close resources of explicitly.

Multi catch block

Until 1.6 version ,Even though Multiple Exceptions having same handling code we have to write a separate catch block for every exceptions, it increases length of the code and reviews readability

```
try{
----------------
----------------
}
catch(ArithmeticException e) {
e.printStackTrace();
}
catch(NullPointerException e) {
e.printStackTrace();
}
catch(ClassCastException e) {
System.out.println(e.getMessage());
}
catch(IOException e) {
System.out.println(e.getMessage());
}
```

To overcome this problem Oracle People introduced "Multi catch block" concept in 1.7 version.

The main advantage of multi catch block is we can write a single catch block , which can handle multiple different exceptions

```
try{
----------------
----------------
}
catch(ArithmeticException | NullPointerException e) {
e.printStackTrace();
}
catch(ClassCastException | IOException e) {
System.out.println(e.getMessage());
}
```

In multi catch block, there should not be any relation between Exception types(either child to parent Or parent to child Or same type , otherwise we will get Compile time error)

```
try{

}catch(ArithmeticException | Exception e){
e.printStackTrace();

}
```

Child class

Parent class

Compile time error

Exception Propagation :

With in a method if an exception raised and if that method doesn't handle that exception, then Exception object will be propagated to the caller then caller method is responsible to handle that exceptions. This process is called Exception Propagation.

Rethrowing an Exception :

To convert the one exception type to another exception type , we can use rethrowing exception concept.

```
class Test {

    public static void main(String[] args) {
        try {
            System.out.println(10 / 0);
        } catch (ArithmeticException e) {
            throw new NullPointerException();
        }
    }
}
```